Architecture 1001: x86-64 Assembly IfExample.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

IfExample.c

```c
int main(){
    int a = -1, b = 2;
    if(a == b){
        return 1;
    }
    if(a > b){
        return 2;
    }
    if(a < b){
        return 3;
    }
    return 0xdefea7;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],0FFFFFFFFh
000000014000100C  mov  dword ptr [rsp],2
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  cmp  dword ptr [rsp+4],eax
000000014000101A  jne  0000000140001023
000000014000101C  mov  eax,1
0000000140001021  jmp  0000000140001048
0000000140001023  mov  eax,dword ptr [rsp]
0000000140001026  cmp  dword ptr [rsp+4],eax
000000014000102A  jle  0000000140001033
000000014000102C  mov  eax,2
0000000140001031  jmp  0000000140001048
0000000140001033  mov  eax,dword ptr [rsp]
0000000140001036  cmp  dword ptr [rsp+4],eax
000000014000103A  jge  0000000140001043
000000014000103C  mov  eax,3
0000000140001041  jmp  0000000140001048
0000000140001043  mov  eax,0DEFEA7h
0000000140001048  add  rsp,18h
000000014000104C  ret
```

IfExample.c

```c
int main(){
    int a = -1, b = 2;
    if(a == b){
        return 1;
    }
    if(a > b){
        return 2;
    }
    if(a < b){
        return 3;
    }
    return 0xdefea7;
}
```

Jcc

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],0FFFFFFFFh
000000014000100C  mov  dword ptr [rsp],2
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  cmp  dword ptr [rsp+4],eax
000000014000101A  jne  0000000140001023
000000014000101C  mov  eax,1
0000000140001021  jmp  0000000140001048
0000000140001023  mov  eax,dword ptr [rsp]
0000000140001026  cmp  dword ptr [rsp+4],eax
000000014000102A  jle  0000000140001033
000000014000102C  mov  eax,2
0000000140001031  jmp  0000000140001048
0000000140001033  mov  eax,dword ptr [rsp]
0000000140001036  cmp  dword ptr [rsp+4],eax
000000014000103A  jge  0000000140001043
000000014000103C  mov  eax,3
0000000140001041  jmp  0000000140001048
0000000140001043  mov  eax,0DEFEA7h
0000000140001048  add  rsp,18h
000000014000104C  ret
```

```
main_0 proc near

var_18= dword ptr -18h
var_14= dword ptr -14h

sub     rsp, 18h
mov     [rsp+18h+var_14], 1
mov     [rsp+18h+var_18], 2 ;
mov     eax, [rsp+18h+var_18]
cmp     [rsp+18h+var_14], eax
jnz     short loc_140001033
```

Ghost of Xmas Future:
Tools you won't get to use today
generate a Control Flow Graph (CFG)
which looks much nicer.
Not that that helps you. Just sayin' :)

```
loc_140001033:
mov     eax, [rsp+18h+var_18]
cmp     [rsp+18h+var_14], eax
jle     short loc_140001043
```

```
loc_140001043:
mov     eax, [rsp+18h+var_18]
cmp     [rsp+18h+var_14], eax
jge     short loc_140001053
```

```
mov     eax, 1
jmp     short loc_140001058
```

```
mov     eax, 2
jmp     short loc_140001058
```

```
mov     eax, 3
jmp     short loc_140001058
```

```
loc_140001053:
mov     eax, 0DEFEA7h
```

```
loc_140001058:
add     rsp, 18h
retn
main_0 endp
```



Jcc - Jump If Condition Is Met

- If a condition is true, the jump is taken. Otherwise it proceeds to the next instruction
- There are more than 4 pages of conditional jump types! Luckily a bunch of them are synonyms for each other.
- JNE == JNZ (Jump if not equal, Jump if not zero, both check if the Zero Flag (ZF) == 0)

Book page 93



Jcc - Jump If Condition Is Met

- If a condition is true, the jump is taken. Otherwise it proceeds to the next instruction
- There are more than 4 pages of conditional jump types! Luckily a bunch of them are synonyms for each other.
- JNE == JNZ (Jump if not equal, Jump if not zero, both check if the Zero Flag (ZF) == 0)

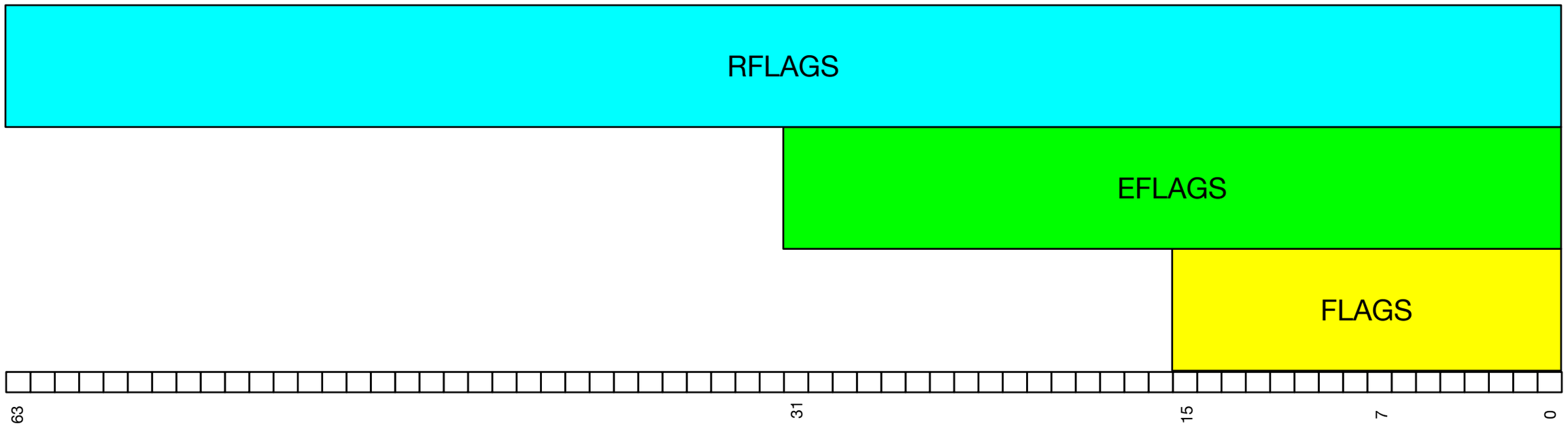
Book page 93

"14"

Jcc - Jump If Condition Is Met

- If a condition is true, the jump is taken. Otherwise it proceeds to the next instruction
- There are more t ges of conditional jump types! L u of them are s
- JN r not zero, both chec Flag (ZF) == 0)

Architecture - RFLAGS



- “In 64-bit mode, EFLAGS is extended to 64 bits and called RFLAGS. The upper 32 bits of RFLAGS register is reserved. The lower 32 bits of RFLAGS is the same as EFLAGS.”

Intel Vol 1 Sec 3.4.3.4 - page 3-18 - Jan 2020 manuals

Architecture - RFLAGS

- RFLAGS register holds many single bit flags. Will only ask you to remember the following for now:
  - Zero Flag (ZF) - Set if the result of some instruction is zero; cleared otherwise.
  - Sign Flag (SF) - Set equal to the most-significant bit of the result, which is the sign bit of a signed integer. (0 indicates a positive value and 1 indicates a negative value.)
- Other flags will be optional material

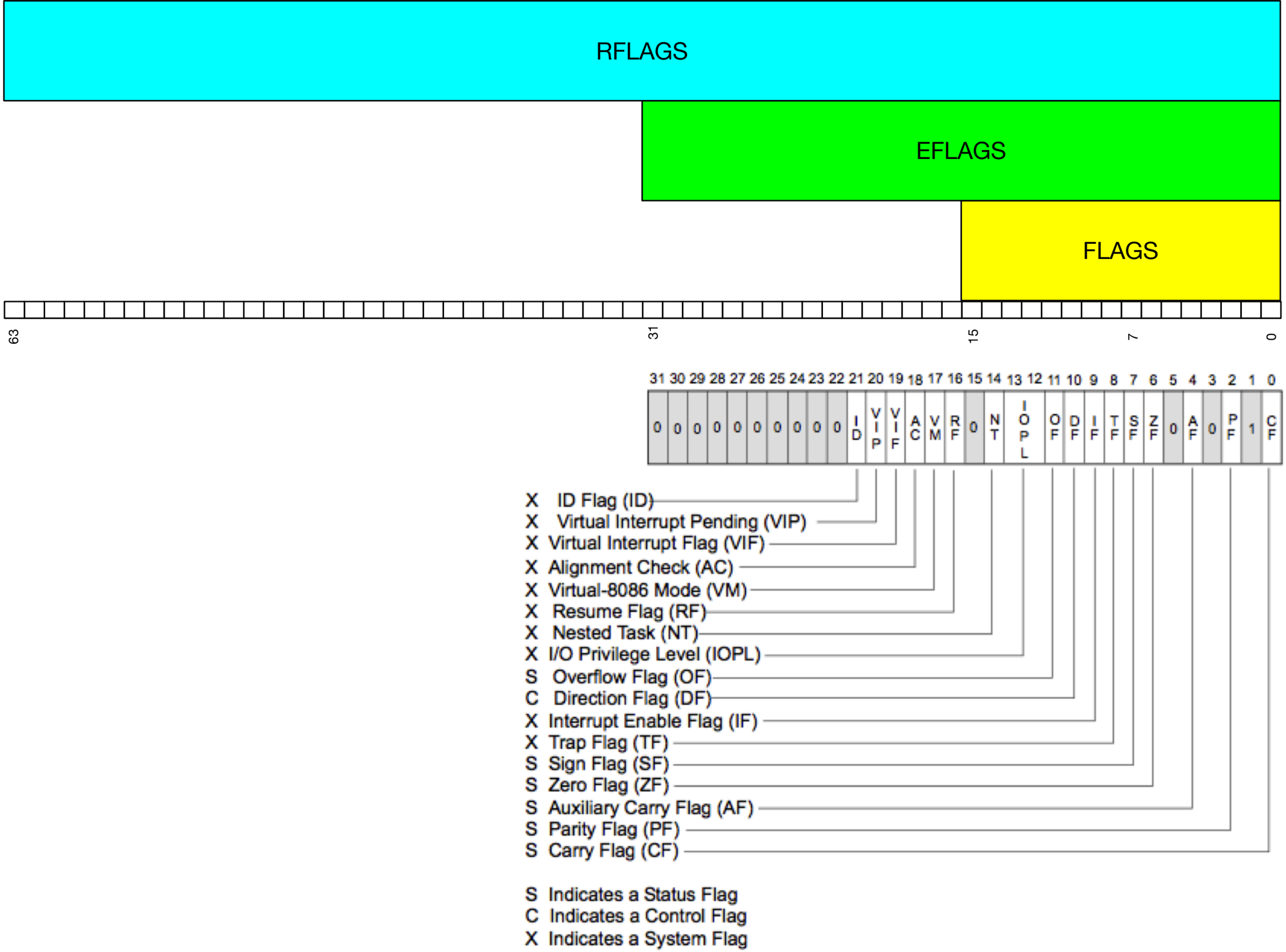


Figure 3-8. EFLAGS Register

This Class

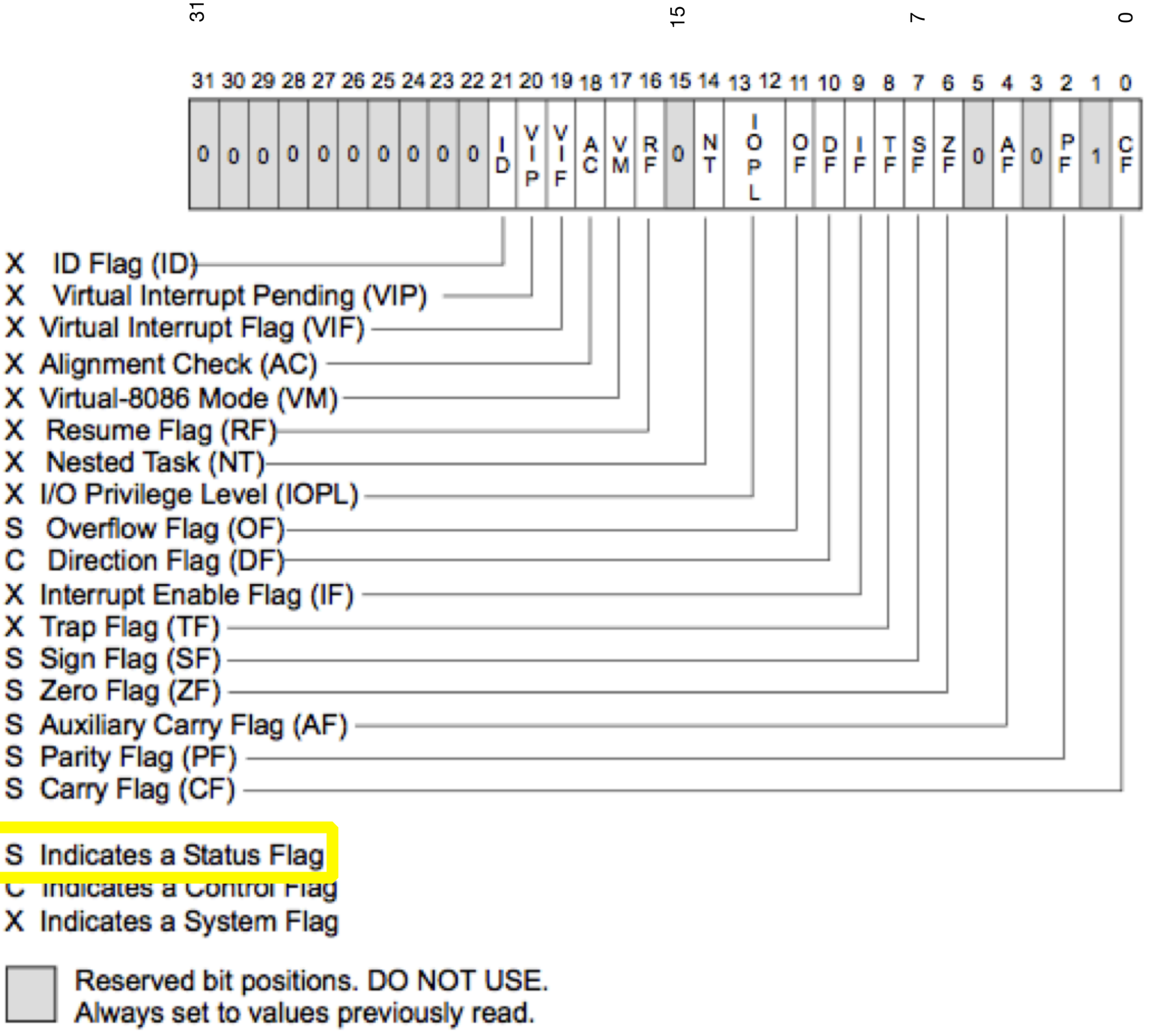


Figure 3-8. EFLAGS Register

This Class

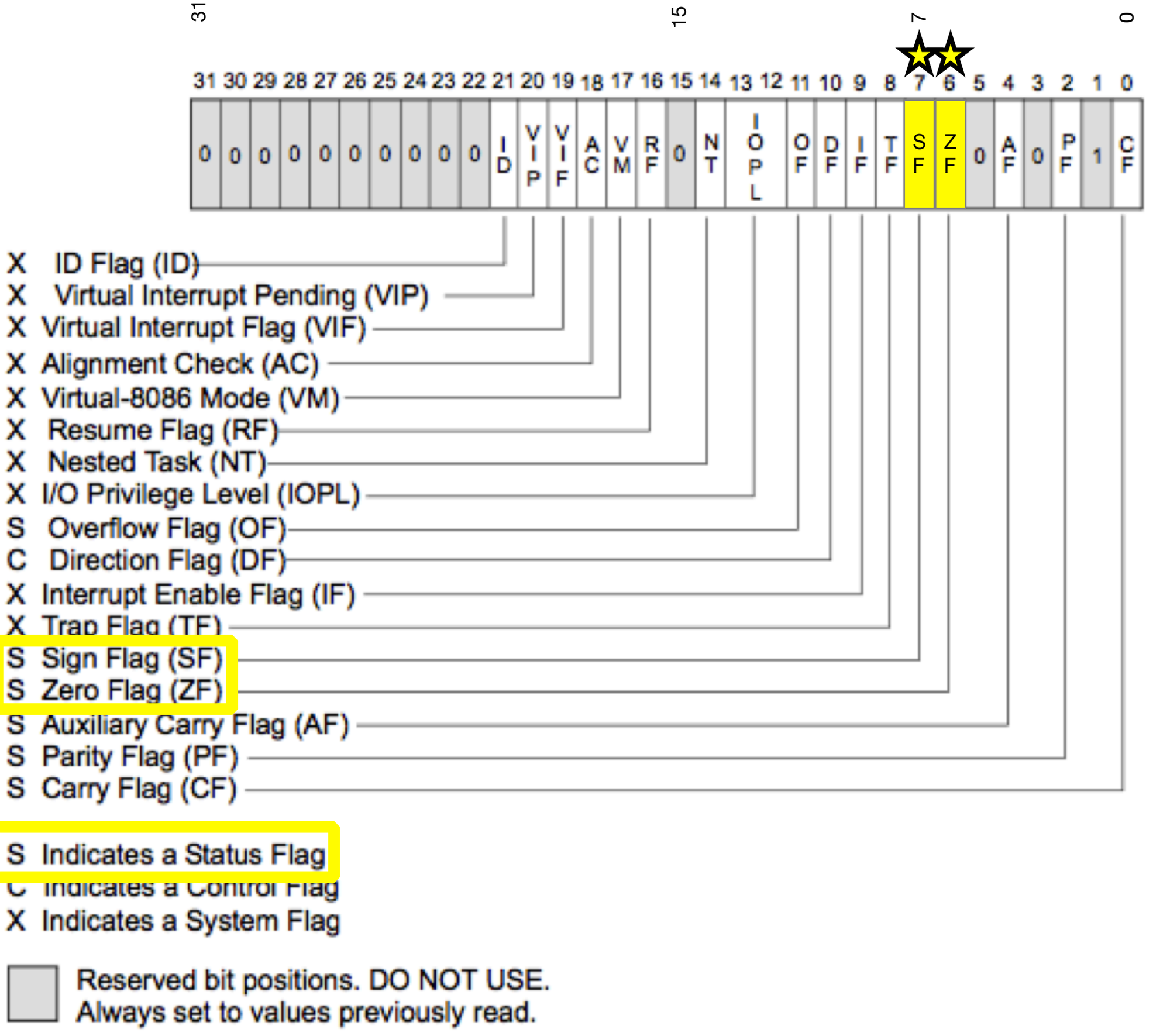


Figure 3-8. EFLAGS Register

This Class

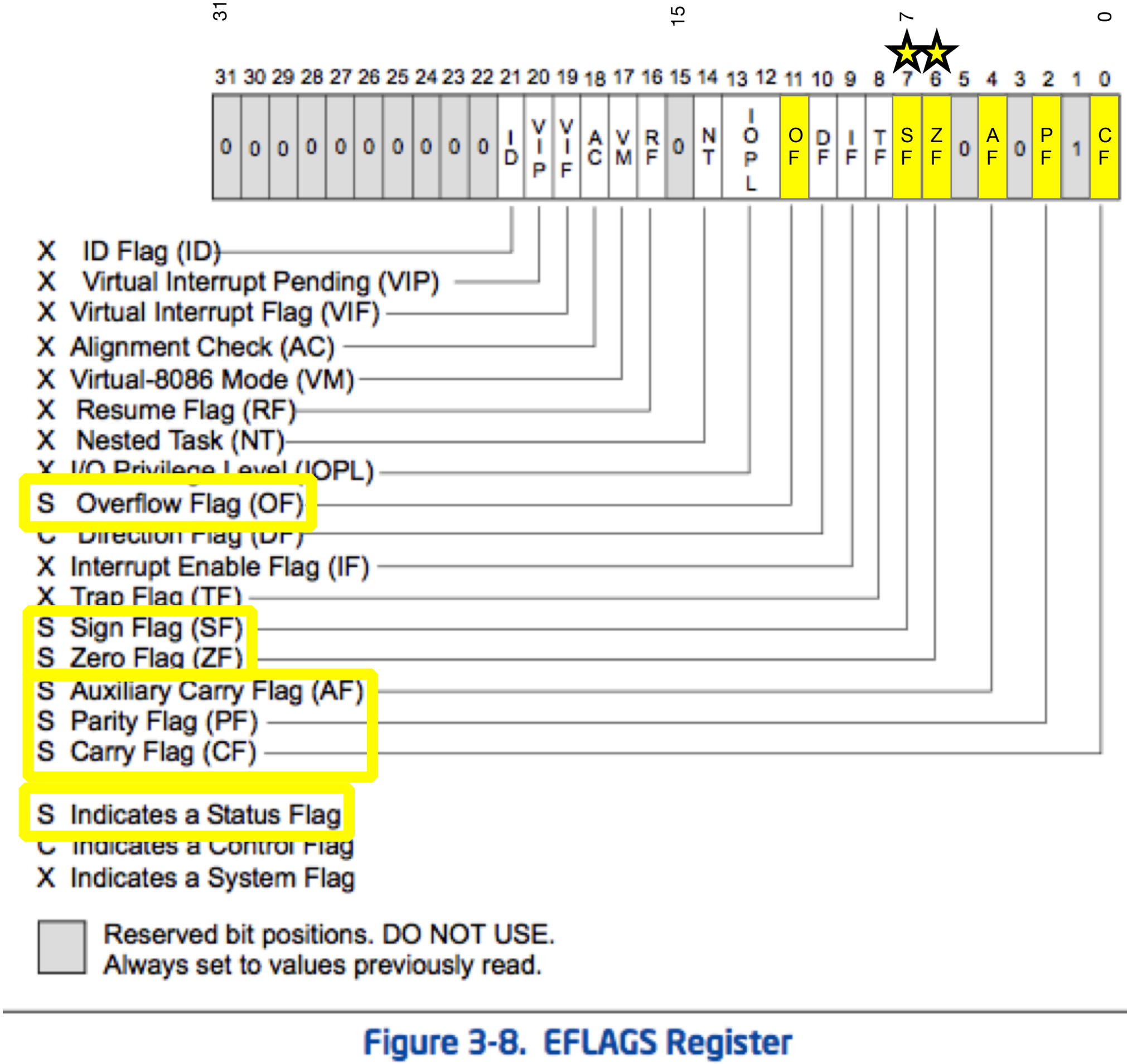


Figure 3-8. EFLAGS Register

This Class

Future Class

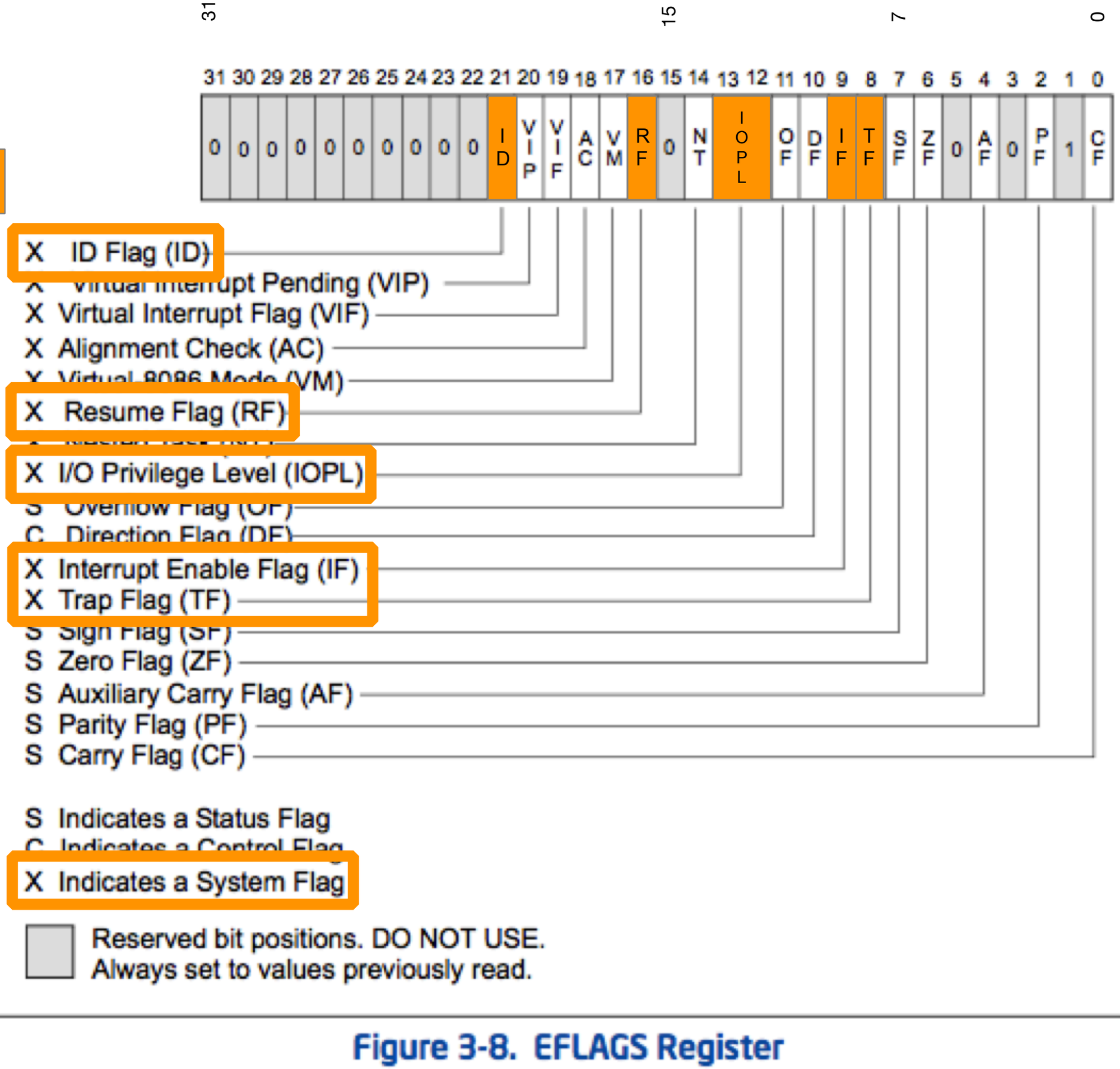
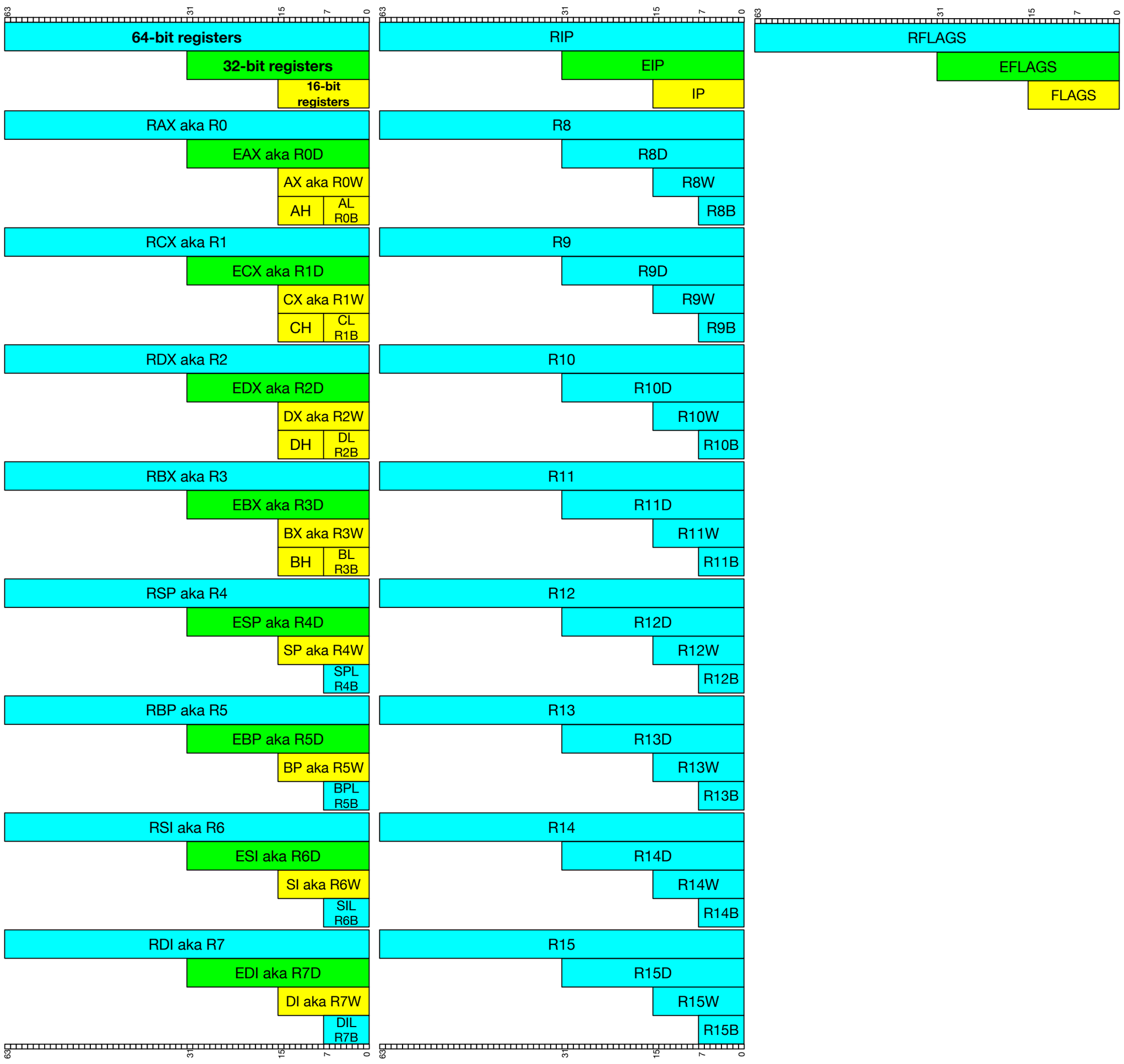


Figure 3-8. EFLAGS Register


63
31
15
7
0
64-bit registers
32-bit registers
16-bit registers
RAX aka R0
EAX aka R0D
AX aka R0W
AH
AL R0B
RCX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
RDX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
RBX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B
RSP aka R4
ESP aka R4D
SP aka R4W
SPL R4B
RBP aka R5
EBP aka R5D
BP aka R5W
BPL R5B
RSI aka R6
ESI aka R6D
SI aka R6W
SIL R6B
RDI aka R7
EDI aka R7D
DI aka R7W
DIL R7B
RIP
EIP
IP
R8
R8D
R8W
R8B
R9
R9D
R9W
R9B
R10
R10D
R10W
R10B
R11
R11D
R11W
R11B
R12
R12D
R12W
R12B
R13
R13D
R13W
R13B
R14
R14D
R14W
R14B
R15
R15D
R15W
R15B
RFLAGS
EFLAGS
FLAGS

Some Notable Jcc Instructions

- JZ/JE: if ZF == 1
- JNZ/JNE: if ZF == 0
- JLE/JNG : if ZF == 1 or SF != OF
- JGE/JNL : if SF == OF
- JBE/JNA: if CF == 1 OR ZF == 1
- JB: if CF == 1
- Note: Don’t get hung up on memorizing which flags are set for what. More often than not, you will be running code in a debugger, not just reading it. In the debugger you can just look at RFLAGS and/or watch whether it takes a jump.

Some Notable Jcc Instructions 2

- Mnemonic translations
- A = above, unsigned notion
- B = below, unsigned notion
- G = greater than, signed notion
- L = less than, signed notion
- E = Equal (same a Z, zero flag set)
- N = Not (like “Not less than:” JNL)

Some Notable Jcc Instructions 3

- Mnemonic translations
- **B** = below, unsigned notion
- **A** = above, unsigned notion
- **N** = Not (like “Not less than:” JNL)
- **G** = greater than, signed notion
- **L** = less than, signed notion
- **E** = Equal (same a Z, zero flag set)

Some Notable Jcc Instructions 3

- Mnemonic translations
- **B** = below, unsigned notion
- **A** = above, unsigned notion
- **N** = Not (like “Not less than:” JNL)
- **G** = greater than, signed notion
- **L** = less than, signed notion
- **E** = Equal (same a Z, zero flag set)



Final Fantasy 7
Crystal Bangle
Unlocked!

Flag Setting

- Before you can do a conditional jump, you need something to set the condition status flags for you
- Typically done with CMP, TEST, or whatever instructions are already inline and happen to have flag-setting side-effects

CMP - Compare Two Operands

- "The comparison is performed by subtracting the second operand from the first operand and then setting the status flags in the same manner as the SUB instruction."
- What's the difference from just doing SUB?
  Difference is that with SUB the result has to be stored somewhere. With CMP the result is computed, the flags are set, but the result is discarded
- Modifies CF, OF, SF, ZF, AF, and PF

Xeno's Guide to (Mostly) Not Caring About Specific Flags

```
cmp dword ptr [rsp+4], eax
jne 0000000140001033

cmp dword ptr [rsp+4], eax
jle 0000000140001043

cmp dword ptr [rsp+4], eax
jae 0000000140001043
```

Xeno's Guide to (Mostly) Not Caring About Specific Flags

```
cmp dword ptr [rsp+4], eax
!= jne 0000000140001033

cmp dword ptr [rsp+4], eax
<= (signed) jle 0000000140001043

cmp dword ptr [rsp+4], eax
>= (unsigned) jae 0000000140001043
```

Xeno's Guide to (Mostly) Not Caring About Specific Flags

```
dword ptr [rsp+4] != eax ?
```

```
dword ptr [rsp+4] <= (signed) eax ?
```

```
dword ptr [rsp+4] >= (unsigned) eax ?
```

Xeno's Guide to (Mostly) Not Caring About Specific Flags

```
cmp dword ptr [rsp+4],eax
jne 0000000140001033
```
Is "`[rsp+4]` **!=** `eax`"?

```
cmp dword ptr [rsp+4],eax
jle 0000000140001043
```
Is "`[rsp+4]` **<=** `eax`"?
(Signed, because "less", not "below")

```
cmp dword ptr [rsp+4],eax
jae 0000000140001053
```
Is "`[rsp+4]` **>=** `eax`"?
(Unsigned, because "above", not "greater")

- Warning: Operands are backwards in AT&T syntax, so you can't just drop the operation in-between the operands!



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

IfExample.c Takeaways

- Conditional logic, like if statements, manifests in assembly as conditional jumps (Jcc). “If condition true, jump there, else fall through”
- Conditions involving in/equality are often checked with a CMP instruction, which is the same thing as a SUB, but it just throws the results away after the relevant RFLAGS bits are set
- The RFLAGS bits are fundamentally what are checked by the Jccs

```
int main(){
    int a = -1, b = 2;
    if(a == b){
        return 1;
    }
    if(a > b){
        return 2;
    }
    if(a < b){
        return 3;
    }
    return 0xdefea7;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],0FFFFFFFFh
000000014000100C  mov  dword ptr [rsp],2
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  cmp  dword ptr [rsp+4],eax
000000014000101A  jne  0000000140001023
000000014000101C  mov  eax,1
0000000140001021  jmp  0000000140001048
0000000140001023  mov  eax,dword ptr [rsp]
0000000140001026  cmp  dword ptr [rsp+4],eax
000000014000102A  jle  0000000140001033
000000014000102C  mov  eax,2
0000000140001031  jmp  0000000140001048
0000000140001033  mov  eax,dword ptr [rsp]
0000000140001036  cmp  dword ptr [rsp+4],eax
000000014000103A  jge  0000000140001043
000000014000103C  mov  eax,3
0000000140001041  jmp  0000000140001048
0000000140001043  mov  eax,0DEFEA7h
0000000140001048  add  rsp,18h
000000014000104C  ret
```

Instructions we now know (15)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/**Jcc (family)**
- **CMP**

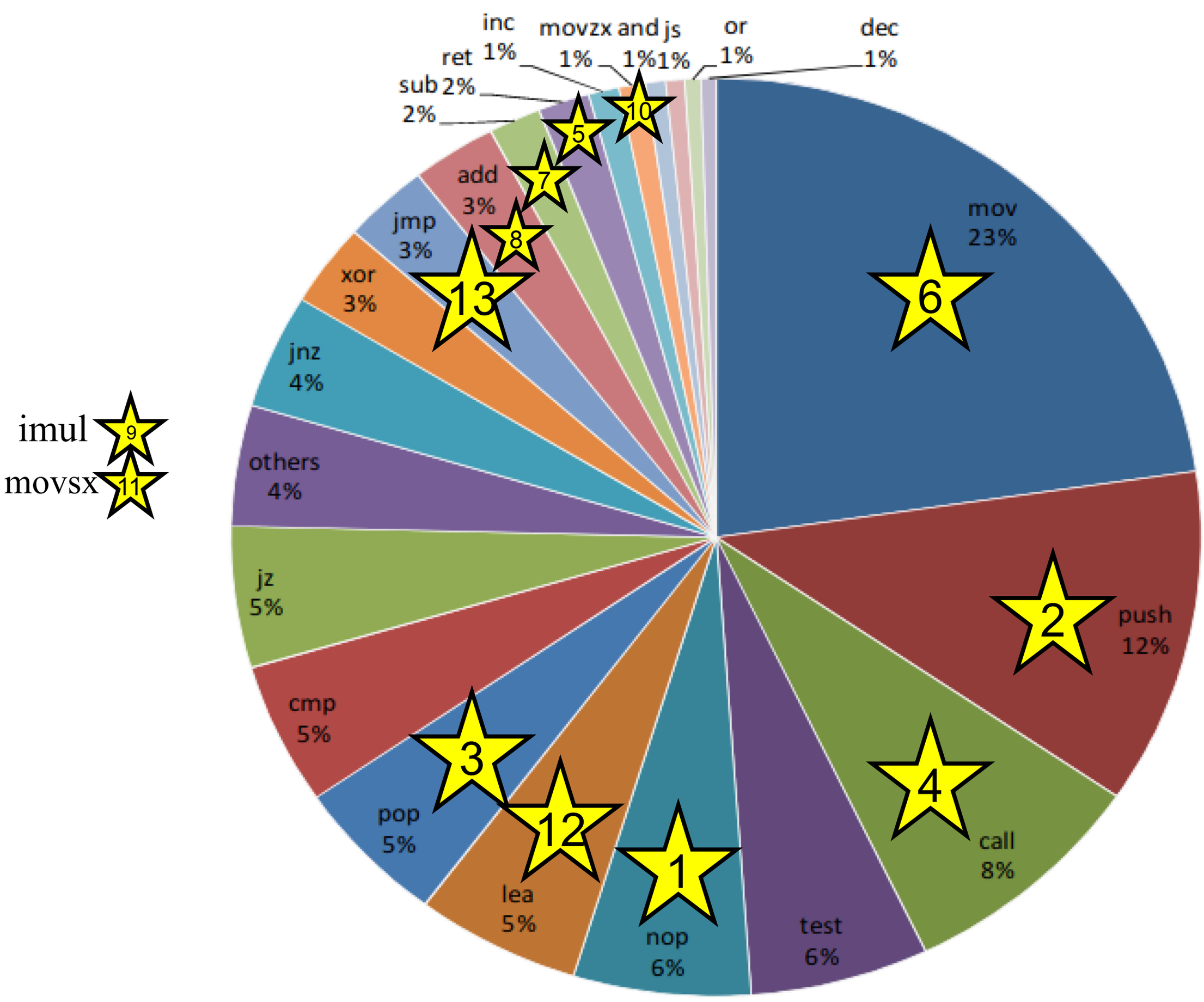


Fig. 2 Web browsers' instruction frequencies

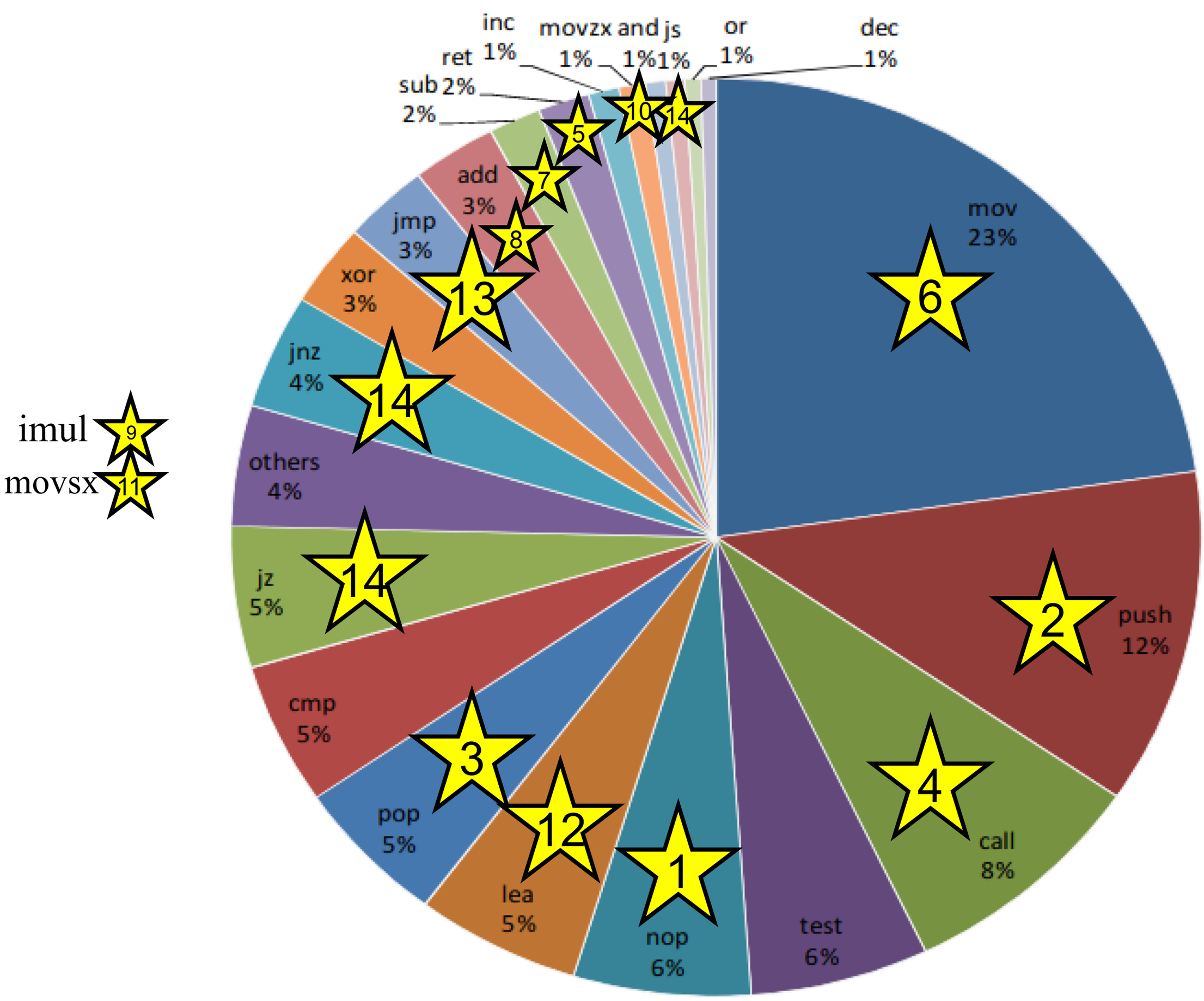


Fig. 2 Web browsers' instruction frequencies

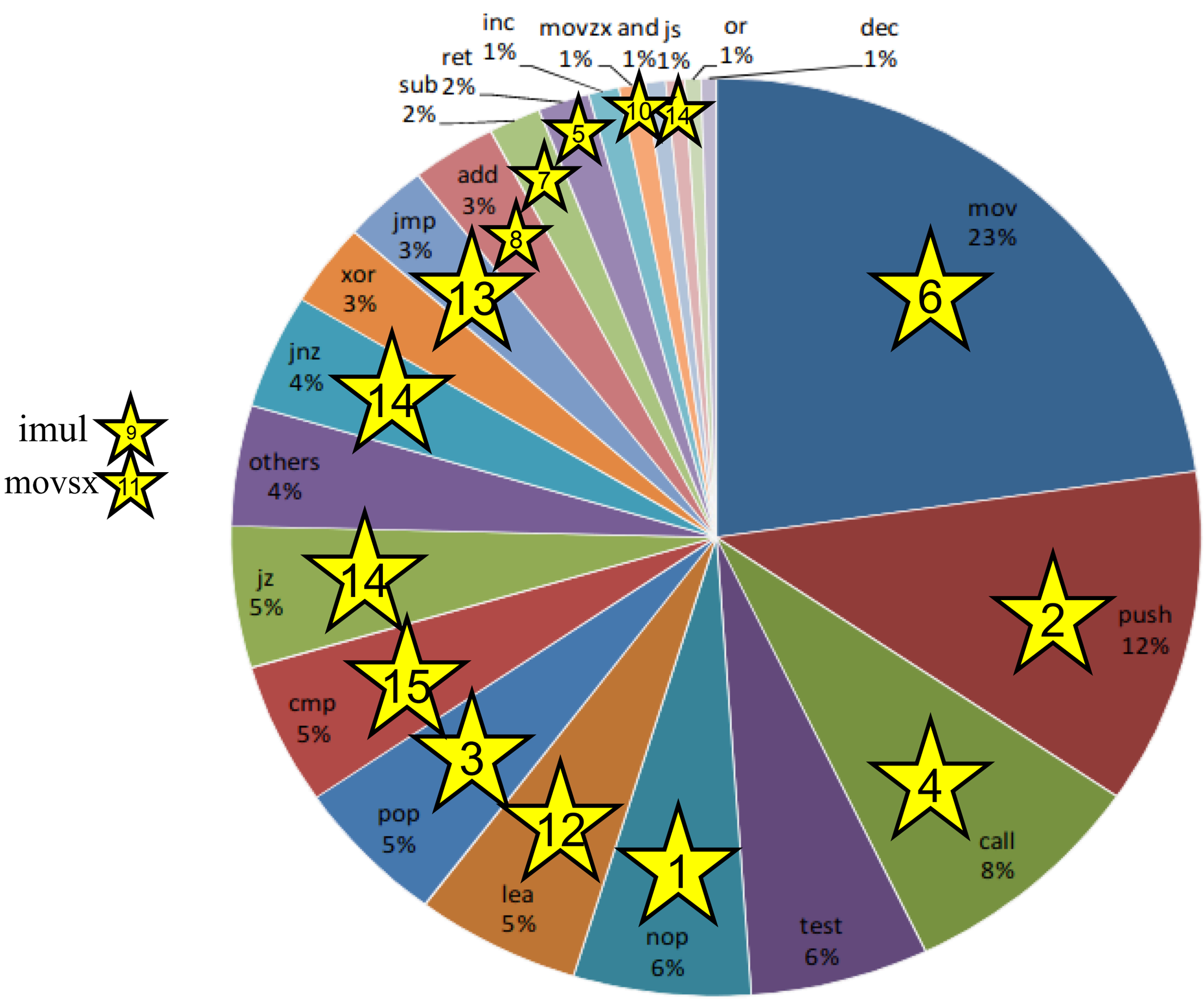


Fig. 2 Web browsers' instruction frequencies